CSC-223
Assignment 8
Work in a Pair

Graph Analyses of DEC Programs

In the previous assignment, we implemented a Control Flow Graph (CFG) and a visitor to generate CFGs from ASTs. For this assignment, we will focus on implementing graph analysis algorithms to extract useful information from these CFGs, which might be leveraged for program optimization and static analysis.

What You Need to Do: Continuing from the Last Assignment

You may continue to work with your partner via the same repository or a new repository for this assignment; the same repository is recommended. Please see the "Git and Github Collaboration" section in the first assignment for details.

What You Need to Do: Adding to the Project Structure

As shown in Figure 1, our analysis code (and tests) will be scattered throughout the project. Both `CFG` and `Digraph` will be modified. Only our tests will result in new files. I would suggest placing your `DiGraph` tests in a `Containers.Tests` namespace.

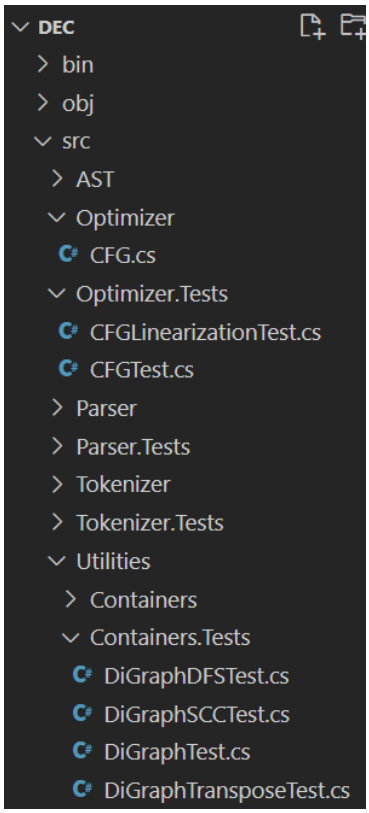

Figure 1: The desired project structure including CFG analysis testing code.

Background: Graph Analysis Algorithms

Graph analysis algorithms are fundamental tools in compiler design and static program analysis. They enable us to reason about program behavior without executing the code, helping to identify potential

optimizations and detect programming errors. For this assignment, we will focus on two general graph algorithms (BFS and DFS) as well as an algorithm to compute *strongly connected components*.

BFS. Recall that Breadth-First Search (BFS) **e**xplores a graph level by level, starting from a source vertex and visiting all neighbors before moving to the next level. Our goal with BFS is to identify the set of reachable and unreachable statements in a CFG. That is, if we begin our analysis from the start `Statement`, we can identify statements in the code that are unreachable and thus would never be executed. Such statements could be optimized away resulting in a smaller footprint for the executable program code.

DFS. We also want to employ a depth-first based analysis on our graphs. Recall that Depth-First Search (DFS) explores a graph by going as deep as possible along each branch before backtracking. DFS is often implemented using a color-marking scheme to track vertex states. Indeed, we will use colors when implementing DFS to enhance debugging.

SCCs. Our last algorithm seeks to identify strongly connected components (SCCs) in a directed graph. A SCC is defined as a set of vertices in a directed graph in which every vertex is reachable from every other vertex in the set. Although we will not be performing these analyses on DEC code, SCCs help identify loops and recursive structures in programs.

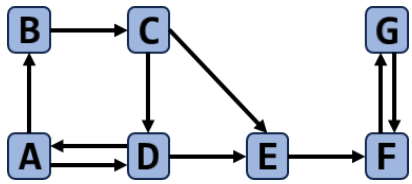

Figure 2: An example directed graph with SCCs {{`A`, `B`, `C`, `D`}, {`E`}, {`F`, `G`}}.

In the graph depicted in Figure 2, we identify three SCCs as follows:

1. Vertices A, `B`, `C`, and `D` form one strongly connected component because there is a path from any one of these vertices to any other; for example, `A->B->C->D->A`.
2. Vertex `E` forms its own strongly connected component (a singleton SCC). While `E` can be reached from `C` or `D`, once you reach `E`, you cannot get back to `C` or `D`.
3. Vertices `F` and `G` form another strongly connected component as they are connected bidirectionally.

Computing SCCs. To compute SCCs, we will implement Kosaraju's algorithm. The algorithm can be stated in a few short steps in Figure **3**.

1. Perform a depth-first search (DFS) on the original graph, keeping track of the order in which vertices complete their DFS recursion. As each vertex finishes, push it onto a stack.
2. Create a transposed graph by reversing all edges from the original graph.
3. Perform a DFS on the transposed graph but start with vertices in the order they appear when popping from the stack (which is the reverse of their finishing order from step 1). That is, begin with the vertex that finished last in the first DFS (the one on top of the stack).
4. Each time you start a new DFS in step 3 (by popping the next vertex from the stack), all vertices you visit during that DFS traversal form one strongly connected component.

Figure 3: Kosaraju's algorithm for computing SCCs in directed graphs.

The elegance of Kosaraju's algorithm lies in how it exploits properties of directed graphs and DFS traversals to find strongly connected components. Two properties are necessary to consider.

Finishing times of Sources and Sinks. After the first DFS, the vertex with the first finishing time must belong to a "***sink SCC***" in the original graph—***an SCC with no edges leading out to other SCCs***.

Imagine the SCCs of a graph as a condensed graph (where each SCC is a single node), this condensed graph must be a directed acyclic graph (DAG). Recall that in any DAG, there must be at least one sink (a node with no outgoing edges). Thus, a DFS exploration ensures that the vertex with the first finishing time must come from one of these sink SCCs.

Flipping Relationships. Another important concept is ***source SCCs***: ***those with no edges coming in from other SCCs***. Based on that description, we can state that source SCCs, because they have outgoing edges, should finish later. When we reverse all edges creating the transpose graph, the SCCs remain the same in the transpose graph; however, importantly, their relationships flip: what was a sink SCC becomes a source SCC and what was a source SCC becomes a sink SCC.

When we conduct the second DFS traversal, we follow the descending order of finishing times from the first DFS, which means we start with vertices that finished last. These vertices typically come from source SCCs in the original graph—components that have no incoming edges from other SCCs. When we transpose the graph by reversing all edges, these former source SCCs transform into sink SCCs, with no outgoing edges to other components. This transformation is crucial because when we start our second DFS from these vertices, the traversal remains contained within its own SCC and cannot 'leak' into other components.

Each time we initiate a new DFS from the next unvisited vertex in our finishing time ordering, we identify a complete and distinct strongly connected component. This elegant property—where the interplay between finishing times and graph transposition naturally isolates SCCs—is what makes Kosaraju's algorithm both correct and efficient. Without this specific ordering based on finishing times, we would not be able to cleanly separate the components during the second traversal.

(These ideas are difficult to digest by simply reading and re-reading the explanation. To gather greater understanding and insight, take a directed graph such as the one in Figure 2 and run the algorithm. Hopefully what you compute agrees with our explanation and SCCs that were described.)

What You Need to Do: BFS

In `CFG.cs`, implement a BFS according to the method signature below. Note how we are returning a tuple consisting of the sets of statements that are (un)reachable from the start statement.

```
public (List<Statement> reachable, List<Statement> unreachable) BreadthFirstSearch()
```

Make sure to test this BFS implementation starting with DEC code that has been (1) parsed, (2) a CFG generated using the visitor from the last assignment, and then (3) any (un)reachable is verified accordingly using BFS.

What You Need to Do: DFS

In `DiGraph.cs`, implement a DFS using the standard color scheme (white, purple, black) according to the method signature below. Observe that the method does not take a starting point as an argument but instead will visit each vertex in the graph in the sequence they are stored in the adjacency list. Recall that the item at the top of the `Stack` was the last vertex to finish.

```
public Stack<T> DepthFirstSearch()
```

Test this functionality in a separate file from other `DiGraph` functionality.

What You Need to Do: Kosaraju's Algorithm

To support Kosaraju's algorithm, implement a `Transpose` method in `DiGraph.cs`, that returns a transposed version of the directed graph. Test this functionality in a separate file from other `DiGraph` functionality.

```
public DiGraph<T> Transpose()
```

Using many of the new methods, implement a method that will compute the SCCs for a `DiGraph`. What is returned is a list of strongly connected components, where each component is a list of vertices.

```
public List<List<T>> FindStronglyConnectedComponents()
```

Again, test this functionality in a separate file from other `DiGraph` functionality.

Test this functionality in a separate file from other `DiGraph` functionality.

What You Need to Do: Verifying the Linear Nature of DEC Programs

In the `CFGLinearizationTest.cs` file, verify that the number of SCCs in a CFG representing a DEC program corresponds directly to the total number of assignment statements and return statements. As an example, consider the DEC program in Figure **4** that was discussed in the description of the last assignment. Observe how the code leads to a linear CFG. Hence, as there are 4 statements, there will be 4 SCCs that would be computed.

```
x := (5)
y := (10)
z := (x + y)
return (z)
```

```
[x := 5] → [y := 10] → [z := x + y] → [return z]
```

Figure 4: An example DEC program and the CFG as a linearization.

What You Need to Do: Testing

Testing specifics were discussed above. You may use an LLM for testing but be warned that you will explicitly be asked about test coverage.

What You Need to Do: Commenting

Comment code consistent with the standards stated in the first assignment: when needed, use an LLM to generate comments then refine as needed.

Submitting: Source Code

You will demonstrate your source code to the instructor, in person. Be ready to demonstrate successful execution of all unit tests and describe all tests and code you were required to implement.